

S/4HANA Live Tenant Bridge — Feature Documentation

Version: 1.6.0

Last Updated: 2026-06-10

Status: Production-Ready (Pilot)

Table of Contents

- [\[Overview\]\(#overview\)](#)
- [\[Architecture\]\(#architecture\)](#)
- [\[Authentication Methods\]\(#authentication-methods\)](#)
- [\[Connection Test — How It Works\]\(#connection-test--how-it-works\)](#)
- [\[Test Execution Pipeline\]\(#test-execution-pipeline\)](#)
- [\[API Reference\]\(#api-reference\)](#)
- [\[Data Model\]\(#data-model\)](#)
- [\[Security Model\]\(#security-model\)](#)
- [\[User Guide — Step by Step\]\(#user-guide--step-by-step\)](#)
- [\[Troubleshooting\]\(#troubleshooting\)](#)
- [\[Changelog\]\(#changelog\)](#)

Overview

The **S/4HANA Live Tenant Bridge** (internally: BYOT — *Bring Your Own Tenant*) allows users to connect their own SAP S/4HANA Cloud sandbox or test system to the Clean-Core.io platform. Once connected, the platform performs **real server-side HTTP validation** against the tenant endpoint during test execution.

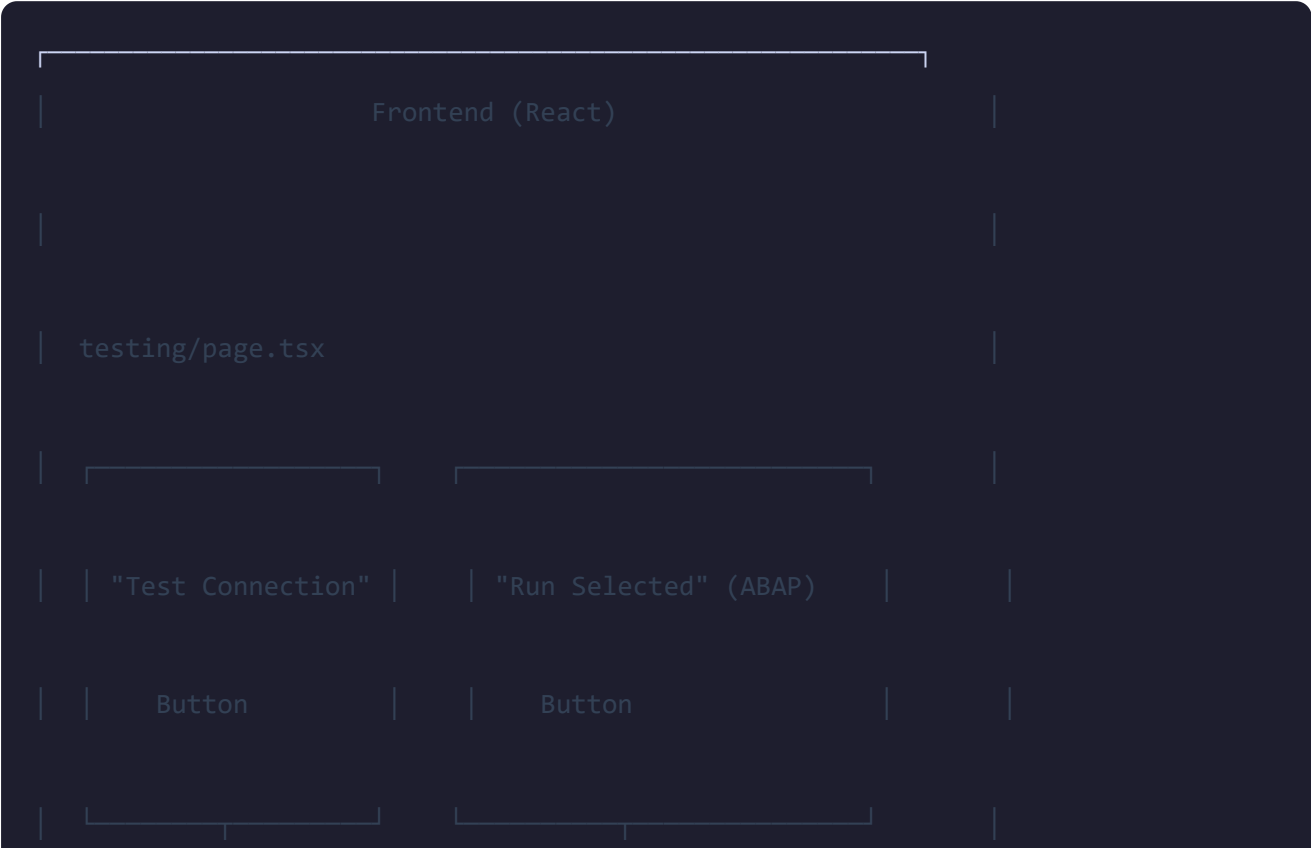
Key Capabilities

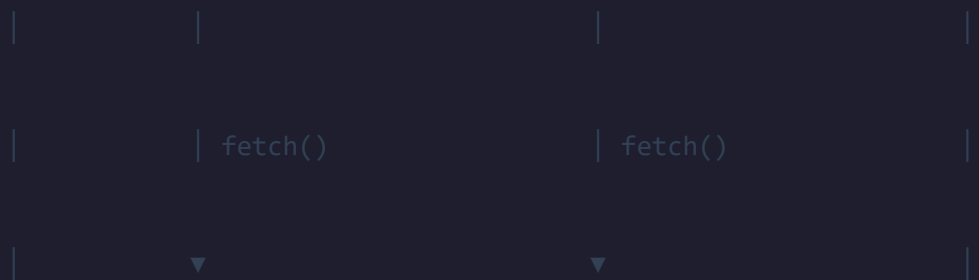
Capability	Description
Real Connection Test	Server-side HTTP HEAD/GET against the tenant URL with full auth handshake
4 Auth Methods	Basic Auth, OAuth 2.0 Client Credentials, SAP API Hub Key, BTP Destination JSON
Live Test Validation	ABAP Cloud test execution validates tenant reachability in real-time
Simulated Fallback	Mock mode clearly labeled as <code>[SIMULATED]</code> when no tenant is connected
Production Safety	Blocks requests to <code>*-api.s4hana.ondemand.com</code> production endpoints

What This Feature Does NOT Do (Yet)

- ❌ Execute actual ABAP Unit test classes on the remote system (requires ADT/Eclipse)
- ❌ Fetch or validate OData `$metadata` schemas
- ❌ Run CREATE/UPDATE/DELETE operations against OData entity sets
- ❌ Support Principal Propagation or SAML-based SSO for test execution

Architecture





Next.js API Routes



/api/test-s4-connection /api/test-s4-connection

(route.ts – 389 lines) (reused by useTestExecution)



1. Parse auth type

2. Build auth headers (Basic/Bearer/APIKey)

3. HTTP HEAD → fallback GET

4. Evaluate HTTP status

5. Return { status, message, httpStatus }



(Customer's system)

File Map

File	Purpose
<code>app/(app)/project/[projectId]/testing/page.tsx</code>	UI: Connection form, test runner, sandbox console
<code>app/api/test-s4-connection/route.ts</code>	API: Real HTTP connectivity test (389 lines)
<code>app/api/run-tests/route.ts</code>	API: Node.js sandbox test runner
<code>hooks/useTestExecution.ts</code>	Hook: Orchestrates test execution (Live vs. Mock)
<code>hooks/useTestGeneration.ts</code>	Hook: AI-powered test case generation via Gemini
<code>lib/types.ts</code>	TypeScript interfaces: <code>S4Config</code> , <code>Project</code>

Authentication Methods

1. Basic Authentication

The standard SAP method for Communication Users.

```
Authorization: Basic base64(username:password)
```

Field	Example
Username	<code>CC_INTEGRATOR</code> or <code>INTEGRATION_USER</code>
Password	Password set in Communication Arrangement

SAP Setup: Communication Management → Communication Arrangements → Create a Communication User with the required Communication Scenarios.

2. OAuth 2.0 Client Credentials

For service-to-service authentication without user context.

```
1. POST tokenUrl → { grant_type: client_credentials }
   Authorization: Basic base64(clientId:clientSecret)

• Response: { access_token: "eyJ..." }

• GET s4Url
   Authorization: Bearer eyJ...
```

Field	Example
Client ID	sb-clone-xxxx-xxxx-...
Client Secret	Generated during Communication Arrangement setup
Token URL	https://.authentication.eu10.hana.ondemand.com/oauth/token

SAP Setup: Communication Arrangements → OAuth 2.0 Details → Copy Client ID, Client Secret, and Token URL.

3. SAP API Hub Sandbox Key

For testing against the SAP Business Accelerator Hub sandbox APIs.

```
APIKey: <your-sandbox-key>
```

Field	Example
API Key	Found at api.sap.com → your profile → Show API Key

4. BTP Destination Service JSON

Paste the full JSON export from SAP BTP Cockpit → Connectivity → Destinations.

```
{
  "Name": "S4HANA_SANDBOX",

  "URL": "https://my426318.s4hana.cloud.sap",

  "Authentication": "BasicAuthentication",

  "User": "CC_INTEGRATOR",

  "Password": "...",

  "ProxyType": "Internet",

  "Type": "HTTP"
}
```

The system auto-detects the authentication type from the JSON:

- **BasicAuthentication** → Basic Auth
- **OAuth2ClientCredentials** → OAuth 2.0 flow
- **PrincipalPropagation** / **NoAuthentication** → URL reachability check only

Connection Test — How It Works

When the user clicks **"Test Connection"**, the following happens:

Sequence

1. Frontend sends POST `/api/test-s4-connection`

Body: `{ url, username, password, authType, btpDestinationJson }`

- API Route validates input:

- URL required and must be HTTPS
- Production domains blocked (*-api.s4hana.ondemand.com)

- Auth headers are built:

- Basic: base64(user:pass) → Authorization header
- OAuth2: Token exchange at tokenUrl → Bearer token
- SAP Hub: password → APIKey header
- BTP: JSON parsed → auto-resolved

- HTTP HEAD request to tenant URL (15s timeout)

- If HEAD fails → fallback to HTTP GET

- HTTP status evaluated:

- 2xx/3xx → "connected" 
- 401/403 → "connected" (reachable but auth rejected) 
- 404 → "failed" (wrong URL path) 
- 5xx → "failed" (server error) 
- Timeout → "failed" (15 seconds exceeded) 

- Response returned to frontend:

```
{ status: "connected"|"failed", message: "...", httpStatus: 200 }
```

Important: What "Connected" Means

A `status: "connected"` response means the **tenant endpoint is reachable** and responded to the HTTP request. This does NOT mean:

- That specific OData services are available
- That the Communication User has the right authorizations
- That the Communication Scenarios are correctly configured

A `401` or `403` is still reported as `"connected"` because it proves the server is reachable — the issue is authentication, not connectivity.

Test Execution Pipeline

ABAP Cloud Projects — Two Modes

Live Mode (`s4Environment === 'live'`)

When a live tenant is configured and connected:

- The hook calls `/api/test-s4-connection` with the saved `s4Config` credentials
- Verifies whether the tenant endpoint is reachable
- Maps test results based on connectivity:
 - **Functional / Business Logic / Transactional** tests → FAILED if tenant unreachable
 - **Structural / Validation** tests → PASSED (don't require live connectivity)
- Generates a `S/4HANA LIVE TENANT VALIDATION REPORT` with:
 - Tenant URL, auth type, connectivity status
 - Per-test PASSED/FAILED with reasons
 - Real HTTP status codes

Mock Mode (`s4Environment === 'mock'` or not configured)

When no live tenant is configured:

- All output prefixed with `[SIMULATED]`
- Tests marked as `[SIMULATED PASS]`

- Report header: `SIMULATED ABAP UNIT TEST REPORT`
- Warning: `*" ⚠️ These results are SIMULATED. No S/4HANA tenant was contacted."*`
- Prompt: `*"Connect a Live Tenant for real validation."*`

Node.js/CAP Projects

Node.js projects use a real sandbox runner (`/api/run-tests`) that executes TypeScript code via `tsx --test` . The tests are AI-generated unit tests that validate the generated application code's internal logic. S/4HANA credentials are passed as environment variables but tests are designed to be isolated (no network calls).

API Reference

`POST /api/test-s4-connection`

Performs a real server-side HTTP connectivity check against the provided S/4HANA tenant URL.

Request Body

```
{
  url: string; // Required. HTTPS URL of the S/4HANA tenant

  username?: string; // Basic: Communication User / OAuth2: Client ID

  password?: string; // Basic: Password / OAuth2: Client Secret / Hub:

  authType: 'basic' | 'oauth2' | 'sap_hub' | 'btp_destination';

  tokenUrl?: string; // OAuth2 only: Token endpoint URL

  btpDestinationJson?: string; // BTP Destination only: Full JSON export
}
```

Response

```
{
  status: 'connected' | 'failed';

  message: string;           // Human-readable explanation

  httpStatus?: number;       // HTTP status code from the tenant

  details?: object;          // Additional context (e.g., tokenUrl for OAuth e

}
```

Status Codes

HTTP	Meaning
200	Success — tenant responded
400	Bad request — missing or invalid parameters
401	OAuth token exchange failed
403	Production domain blocked
502	Connection to tenant failed (timeout, DNS, SSL)
500	Internal server error

Examples

Basic Auth — Successful:

```
curl -X POST /api/test-s4-connection \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://my426318.s4hana.cloud.sap",
    "username": "CC_INTEGRATOR",
```

```
"password": "SecurePass123!",  
  
"authType": "basic"  
  
}'
```

Response:

```
{ "status": "connected", "message": "Conne
```

Wrong Credentials:

```
# Response:  
{ "status": "connected", "message": "Endpo
```

Wrong URL:

```
# Response:  
{ "status": "failed", "message": "Endpoint
```

Data Model

S4Config Interface

```
// lib/types.ts  
  
interface S4Config {  
  
    url?: string; // e.g. "https://my426318.s4hana.cloud.sap"
```

```
username?: string;      // Communication User or Client ID

password?: string;      // Password or Client Secret

authType?: 'basic' | 'oauth2' | 'sap_hub' | 'btp_destination';

btpDestinationJson?: string; // Full BTP Destination JSON export

}
```

Project Interface (relevant fields)

```
interface Project {
  // ... other fields ...

  s4Deployment?: 'public' | 'private'; // Public Cloud vs. Private Cloud

  s4Environment?: 'mock' | 'live';     // Mock (simulated) vs. Live (real tenant)

  s4Config?: S4Config;                 // Tenant connection credentials

}
```

Firestore Storage

Credentials are stored in the project document at `projects/{projectId}` :

```
projects/
  {projectId}/

  s4Config: {

    url: "https://...",
```

```
    username: "CC_INTEGRATOR",

    password: "...",           ← stored as-is (see Security section)

    authType: "basic"

  }

  s4Environment: "live"
```

Security Model

Current Safeguards

Control	Implementation
HTTPS Only	URL validation rejects non-HTTPS URLs
Production Block	Domains matching <code>*-api.s4hana.ondemand.com</code> are blocked server-side
Timeout	15-second timeout on tenant requests, 12-second timeout on OAuth token exchange
Server-Side Execution	All HTTP requests are made from the Next.js API route, never from the browser
No Write Operations	Only HTTP HEAD/GET — no POST/PUT/DELETE against the tenant
Error Sanitization	Error messages are truncated to 200 chars to prevent information leakage

Known Limitations & Risks

 **Credential Storage:** Passwords and Client Secrets are currently stored **in plaintext** in Firestore. In a production deployment, these should be encrypted at rest or stored in a

secrets manager (e.g., Google Secret Manager, SAP BTP Credential Store).

⚠️ **No Rate Limiting:** The `/api/test-s4-connection` endpoint has no rate limiting. Repeated calls could trigger IP-based blocking on the customer's SAP system.

⚠️ **CORS Proxy:** The API route acts as a server-side proxy. While this is necessary for CORS, it means the Clean-Core.io server has transient access to the credentials.

User Guide — Step by Step

Prerequisites

- **SAP S/4HANA Cloud** sandbox or test system (not production!)
- **Communication User** with appropriate Communication Scenarios configured
- The system must be reachable from the internet (no VPN/firewall blocking)

Step 1: Open the Testing Page

Navigate to your project → **Stage 5: Testing** → scroll to the **S/4HANA Live Tenant Bridge** panel.

Step 2: Enter Tenant URL

Enter your S/4HANA Cloud system URL:

```
https://my426318.s4hana.cloud.sap
```

Do **not** include trailing paths like `/sap/opu/odata/`. Just the base URL.

Step 3: Select Authentication Method

Choose one of:

- **Basic Auth** — for Communication Users (most common)
- **OAuth 2.0** — for service-to-service with Client Credentials
- **SAP API Hub** — for api.sap.com sandbox testing
- **BTP Destination** — paste your full Destination JSON

Step 4: Enter Credentials

- **Basic Auth:** Enter Communication User name + password
- **OAuth 2.0:** Enter Client ID as username, Client Secret as password

Step 5: Test Connection

Click "**Test Connection**". You will see:

-  [SUCCESS] Connection successful via Basic Auth. HTTP 200 — your system is reachable and credentials are valid
-  [SUCCESS] Endpoint reachable (HTTP 401). Authentication credentials were rejected. — system reachable but wrong credentials
-  [ERROR] Connection timed out after 15 seconds. — system not reachable

Step 6: Save & Run Tests

Click "**Save Connection**" to persist your credentials. Then:

- Generate a test suite (or use existing tests)
- Select test cases and click "**Run Selected**"
- Tests now validate against your real tenant
- Report shows **S/4HANA LIVE TENANT VALIDATION REPORT** with actual connectivity status

Troubleshooting

"Connection timed out after 15 seconds"

Cause: The S/4HANA system is not reachable from the internet. **Solutions:**

- Verify the URL is correct (no typos, no trailing path)
- Check if the system requires VPN access (not supported in pilot)
- Try opening the URL directly in a browser to verify it loads
- Check if SAP has scheduled maintenance

"Authentication credentials were rejected (HTTP 401)"

Cause: The username/password combination is wrong, or the Communication User is locked. **Solutions:**

- Verify the Communication User name (case-sensitive)
- Reset the password in Communication Arrangements
- Check if the user is locked (too many failed attempts)
- Ensure the Communication Arrangement is active

"Endpoint returned HTTP 404"

Cause: The base URL path is wrong. **Solutions:**

- Use only the base URL: `https://my426318.s4hana.cloud.sap`
- Don't include OData paths like `/sap/opu/odata/...`

"Production tenant API endpoints are blocked"

Cause: The URL contains `*-api.s4hana.ondemand.com`, which is a production API endpoint. **Solutions:**

- Use your sandbox/test system URL instead
- This is a safety measure to prevent accidental production access

"OAuth 2.0 token exchange failed"

Cause: The Token URL, Client ID, or Client Secret is incorrect. **Solutions:**

- Verify the Token URL is correct (check Communication Arrangement → OAuth 2.0 Details)
- Ensure the Client ID starts with `sb-clone-`
- Re-generate the Client Secret if in doubt

Tests show "[SIMULATED]" instead of real results

Cause: The project is in Mock mode (no live tenant configured). **Solutions:**

- Set `s4Environment` to `live` in the project settings
- Configure and test the S/4HANA connection first
- Save the connection before running tests

Changelog

v1.6.0 (2026-06-10)

- **BREAKING:** Connection Test now performs real HTTP requests instead of client-side simulation
- **NEW:** ABAP test execution differentiates between Live mode (real validation) and Mock mode (labeled `[SIMULATED]`)
- **FIX:** Tests no longer hardcode all results as PASSED
- **FIX:** Mock mode clearly communicates that results are simulated
- Resolves Michele Mangieri's feedback on fake test results

v1.5.x

- Initial S/4HANA Live Tenant Bridge UI
 - Connection form with 4 auth types
 - API route (`/api/test-s4-connection`) implemented but not wired to frontend
 - All test results were simulated (hardcoded PASSED)
-